# COMP 3700A: Contributing to Open Source

# Continuous Integration and Deployment

Charlotte Curtis

November 9, 2026

# Housekeeping

- Do we have notetakers for today?
  - We might need to come up with some additional topics
- Have you signed up for presentations?
- How is part two of your project coming along?

# Topic Overview

- Why conduct a code review?
- How to do a good code review
- How to respond to code review

Primary reference: Google's Engineering Practices Documentation

# Code review: the "why"

- After submitting a PR, your code will (likely) be built and tested in CI
  - Fix any problems identified by this process, and (force) push fixes
- CI is not enough! You need a human to check if the code:
  - Actually solves the problem
  - Is not excessively complicated or redundant
  - Is readable and follows project conventions



***What are some ways that code reviews might be different in FOSS?***

# Code review: the ""why""

From the Google guide:

> *In general, reviewers should favor approving a CL once it is in a state where it definitely improves the overall code health of the system being worked on, even if the CL isn't perfect.*

- Reviewers are ultimately responsible for changes allowed in a code base
- Reviewers also serve as mentors/teachers

CL = "changelist", AKA patch, pull/merge request, etc

# Code review: the "how"

A good reviewer will:

- Start with a broad view of whether the change is desireable
- Read line by line to get a sense of what the code is doing, and whether the approach fits with the project
- Test the changes by checking out a local branch and rebuilding
- Comment on specific lines to ask for clarification or make suggestions
- Be nice! Provide constructive criticism where necessary, but don't be mean

> *Good reviews take **a lot** of time*



***Can you find some examples of good and "bad" reviews?***

# Code review: potential outcomes

1. Closed as not relevant, superseded, against policy, etc
2. Closed with a recommendation to resubmit as multiple
3. Major revisions requested
4. Minor feedback provided, changes may or may not be requested
5. Accepted and merged

> *Have I forgotten something?*

# Responding to a code review

- If you have outcomes 4-5, great! Eventually, you will encounter 1-3.
- Accepting criticism is a skill that takes practice
- If something makes you upset, cool off before responding
- If the review was rude, be the bigger person. What was their actual message, regardless of tone?
- Ultimately, assume that you both want the same outcome

# Various common scenarios

- If it's clear that the reviewer did not understand your code, try to make the code itself more clear — chances are others will be confused as well
- If you do not understand a reviewer's comments, ask for clarification. It can be helpful to rephrase their comments for a simple yes/no answer.
- If a reviewer has a good suggestion, implement it
- If you disagree with a reviewer's comment, be polite in explaining why you chose your approach

> *It's all about communication and trust!*

# Coming up next

- No class Wednesday, Rememberance Day
- Lab activity: how to check out and build someone else' PR
- Project part 2 due Friday